AI Engineer Hiring Assignment – LLM-Powered CRM Assistant with Local RAG and AWS Bedrock Integration

Overview

This assignment evaluates your ability to design and implement a practical LLM-powered assistant for internal business use.

You will build a prototype assistant that can understand company data, retrieve relevant information, reason across multiple sources, and generate grounded responses for internal teams.

The system must include:

1. A local RAG assistant using a local open-source LLM.
2. An AWS Bedrock execution mode.
3. Source-grounded answers.
4. A simple user interface or API.
5. Clear documentation and demo readiness.
6. A cost-conscious AWS design.

The goal is to demonstrate practical AI engineering ability, not to build a full production platform. Your solution should show strong architecture, reliable retrieval, thoughtful use of LLMs, security awareness, and the ability to explain technical decisions clearly.

AI Coding Tools

You may use AI coding assistants such as Codex, Claude, Cursor, Copilot, or similar tools.

Use of these tools is allowed, but the final solution must be fully understood by you.

During the demo, you should be able to explain your implementation, architecture, trade-offs, and limitations. Code that works but cannot be explained will not be considered sufficient.

Dataset

You will receive a dataset representing internal company data.

The dataset may include structured and unstructured information such as:

1. CRM customer records
2. Leads and sales data
3. Support tickets

4. Sales notes
5. Meeting notes
6. Internal documentation
7. Product descriptions
8. FAQs and policies
9. Email threads
10. Proposal and implementation documents

Dataset Structure

The dataset will be organized in folders similar to the following:

```
crm_records/
sales/
tickets/
emails/
documents/
files/
```

The system should be able to process the dataset and support future additions without requiring major changes.

Task Objective

Build a system that allows a user to interact with the dataset using natural language.

The assistant should retrieve relevant information from the dataset and generate useful, grounded responses.

The system should be able to:

1. Answer questions using internal company documents.
2. Summarize customer accounts.
3. Draft responses to customer emails.
4. Retrieve relevant information from CRM records.
5. Combine information from multiple sources when required.
6. Provide supporting sources for answers.
7. Clearly state when the available data is insufficient.
8. Avoid hallucinating unsupported information.
9. Handle incomplete, conflicting, or ambiguous information.
10. Support both local LLM mode and AWS Bedrock mode.

Example Use Cases

The assistant should support use cases such as:

1. Summarize the history of a customer before a meeting.
2. Explain what channels or features a product supports.
3. Draft a reply to a customer email.
4. Find the support SLA for a specific issue type.
5. Identify customers with open critical tickets.
6. Summarize recent sales notes for a lead.
7. Generate a short account brief before a sales call.
8. Compare information from CRM records, tickets, emails, and documents.
9. Identify missing information needed before responding to a customer.
10. Explain which documents support a given answer.

Part 1: Local RAG Assistant

Requirements

The base system must run locally.

You must build a working local RAG assistant that can:

1. Load and process the provided dataset.
2. Work with both structured and unstructured files where possible.
3. Retrieve relevant information from the dataset.
4. Generate answers using a local open-source LLM.
5. Ground answers in retrieved context.
6. Show the sources used for each answer.
7. Provide either a simple web UI, command-line interface, or API.
8. Allow the dataset index to be rebuilt when files are added or changed.
9. Provide clear local setup and run instructions.

Local Model Requirement

The local version must run without AWS and without external hosted LLM APIs.

External hosted APIs such as OpenAI, Anthropic, Gemini, or similar services are not allowed for the local mode.

You may use open-source models running locally, for example:

1. Llama
2. Mistral
3. Phi
4. Qwen

5. Similar local open-source models

You may choose the local model and tooling. The quality of the architecture, retrieval, grounding, and explanation is more important than using the largest model.

Retrieval Requirement

The system must include a retrieval layer that can find relevant information from the provided dataset.

You may choose the retrieval approach and storage technology.

The retrieval approach should support source tracking so that generated answers can reference the documents or records used.

A strong solution will show thoughtful handling of both structured data and unstructured text.

Part 2: AWS Bedrock Integration

Objective

Extend the system with an AWS Bedrock mode.

The same application should be able to run in:

1. Local mode
2. Bedrock mode

The local mode must remain fully functional without AWS.

The Bedrock mode should use Amazon Bedrock for LLM generation while preserving the same overall assistant behavior, including retrieval, grounding, source references, and refusal when the dataset does not contain enough information.

You may choose the architecture and AWS services needed for your implementation, but the design must remain simple, temporary, and cost-conscious.

Bedrock Requirements

The Bedrock integration must:

1. Use Amazon Bedrock for response generation.
2. Use retrieved context from the dataset.
3. Produce grounded answers with sources.
4. Handle errors and unavailable AWS access gracefully.
5. Avoid sending unnecessary data to Bedrock.

6. Avoid exposing secrets or credentials.
7. Keep behavior consistent with local mode where possible.

Direct use of hosted LLM APIs outside Amazon Bedrock is not allowed in the application runtime.

AWS Cost Constraint

The AWS part must be designed to cost less than €3 for the full assignment demo.

This is a hard requirement.

Avoid any service or configuration that creates unnecessary fixed cost or long-running infrastructure.

The README must include:

1. Estimated AWS cost for the demo.
2. AWS resources used.
3. Any assumptions behind the estimate.
4. Cleanup instructions.
5. A clear warning about resources that must be deleted after the demo.

All AWS resources created for the assignment should be temporary and easy to remove.

Part 3: Expected System Behavior

Grounded Answers

The assistant must answer using the dataset, not general knowledge.

Each answer should include:

1. The generated answer.
2. The source documents, records, or files used.
3. A clear statement when the answer cannot be found in the dataset.

If the dataset does not contain enough information, the assistant should say so rather than guessing.

Multi-Source Reasoning

Some questions may require information from more than one source.

For example, a customer summary may require CRM records, sales notes, support tickets, and email history.

The assistant should synthesize the information into a useful answer instead of simply pasting retrieved text.

Structured and Unstructured Data

The dataset may contain both structured records and unstructured documents.

The system should handle both where possible.

The README should explain how your solution handles different data types and what limitations remain.

Conflicting or Missing Information

The assistant should handle cases where:

1. Two sources disagree.
2. A customer name is ambiguous.
3. A question cannot be answered from the dataset.
4. Information is incomplete.
5. The answer requires assumptions.

In these cases, the assistant should clearly explain the uncertainty.

Prompt-Injection Awareness

The dataset may contain text that looks like instructions to the assistant.

The assistant should treat such text as document content, not as trusted system instructions.

The README should briefly explain how your solution reduces this risk.

Part 4: Evaluation

Include a small evaluation approach to demonstrate that the system works.

Your evaluation should cover:

1. Questions answerable from the dataset.
2. Questions requiring multiple sources.
3. Questions where the correct behavior is refusal.
4. Questions involving customer summaries.
5. Questions involving email drafting.
6. Questions involving structured records.
7. Questions involving conflicting or incomplete information.
8. Questions testing source grounding.

Include enough examples to show the system's behavior clearly.

The evaluation does not need to be perfect, but it should be repeatable and useful for understanding the system's strengths and weaknesses.

Part 5: Observability and Debugging

The system should provide enough visibility to understand how an answer was produced.

At minimum, it should be possible to inspect:

1. The user question.
2. The retrieved sources.
3. The selected LLM mode.
4. Errors or failed requests.
5. Basic latency or runtime behavior.
6. Bedrock usage or estimated cost when AWS mode is used.

Do not log secrets, credentials, or unnecessary sensitive content.

Part 6: Security Requirements

The implementation should demonstrate basic security awareness.

You should:

1. Avoid committing secrets.
2. Use environment variables or another safe configuration approach.
3. Provide an example configuration file without real credentials.
4. Use limited AWS permissions where applicable.
5. Avoid exposing arbitrary local files through the application.
6. Avoid sending unnecessary customer data to Bedrock.
7. Document what data is sent to AWS in Bedrock mode.
8. Avoid logging sensitive content unnecessarily.

Allowed Tools

You may use tools and frameworks such as:

1. Python
2. HuggingFace Transformers
3. Ollama or llama.cpp
4. LangChain or LlamaIndex
5. FAISS, Chroma, SQLite, or similar retrieval/indexing tools
6. FastAPI, Flask, Streamlit, or a simple CLI

7. Amazon Bedrock
8. AWS SDKs
9. Other AWS services if justified and kept within the cost limit
10. AI coding assistants such as Codex, Claude, Cursor, Copilot, or similar tools

You may choose the tools and architecture, but you must explain your choices.

Not Allowed

The following are not allowed:

1. Using OpenAI, Anthropic, Gemini, or other hosted LLM APIs directly in the application runtime.
2. Replacing the required local mode with a hosted API.
3. Hardcoding answers to expected questions.
4. Returning answers without sources.
5. Ignoring the AWS cost constraint.
6. Committing credentials, API keys, or secrets.
7. Using expensive always-on infrastructure without strong justification.
8. Requiring undocumented manual setup steps.
9. Submitting code that cannot be explained during the demo.

Deliverables

Submit:

1. Working code repository.
2. README with setup and run instructions.
3. Local mode instructions.
4. Bedrock mode instructions.
5. Short architecture explanation.
6. Explanation of retrieval and grounding approach.
7. Example questions and outputs.
8. Evaluation examples or evaluation script.
9. AWS cost estimate.
10. AWS cleanup instructions.
11. Live demo during the interview.

README Requirements

The README should include:

1. Project overview.
2. Setup instructions.
3. How to run local mode.
4. How to run Bedrock mode.
5. Model and tool choices.

6. Retrieval strategy.
7. Handling of structured and unstructured data.
8. Grounding and source citation approach.
9. Prompt-injection risk handling.
10. AWS architecture explanation.
11. AWS cost estimate.
12. Cleanup instructions.
13. Known limitations.
14. Future improvements.

Expected Demo

During the interview, be ready to demonstrate:

1. Running the assistant.
2. Asking questions against the dataset.
3. Showing retrieved sources.
4. Summarizing a customer account.
5. Drafting a customer email response.
6. Switching between local mode and Bedrock mode.
7. Explaining the architecture.
8. Explaining AWS cost control.
9. Explaining how AWS resources can be removed.
10. Explaining limitations and possible improvements.
11. Handling a small change to the dataset or a new question during the demo.

Evaluation Criteria

The assignment will be evaluated based on:

1. System architecture and design.
2. Retrieval quality.
3. RAG implementation quality.
4. Effective local LLM usage.
5. Bedrock integration.
6. Grounded answers with sources.
7. Handling of missing or conflicting information.
8. Handling of structured and unstructured data.
9. Security and prompt-injection awareness.
10. AWS cost awareness.
11. Code clarity and maintainability.
12. Practical usefulness of the solution.
13. README quality.
14. Ability to explain decisions during the demo.
15. Ability to debug or adapt the system when asked.

Time Expectation

The assignment is expected to take approximately 2-3 focused days.

Focus on building a working, understandable prototype rather than a production-ready system.

The AWS integration should be small, controlled, and cost-aware.

Do not spend time building a large cloud platform. A clean, practical solution with good engineering decisions is preferred.

Important Notes

1. Local mode is required.
2. Bedrock mode is required.
3. The total AWS cost for the demo should stay under €3.
4. All AWS resources created for this assignment should be deleted after completion.
5. Do not use paid hosted LLM APIs other than Amazon Bedrock for the AWS integration.
6. Do not commit secrets or credentials.
7. Answers must be grounded in the provided dataset.
8. Use of AI coding tools is allowed, but the final solution must be fully explainable.
9. The solution should prioritize correctness, maintainability, and debuggability.
10. The AWS setup should be temporary and easy to remove.